

ServiceFlow

Gestão de ordens de serviço para assistência técnica

Tipo	Documento de Requisitos de Software
Versão	2.1
Data	21/09/2026
Objetivo	Especificar um produto de portfólio pequeno e aplicável a assistência técnica de computadores, celulares, eletrônicos ou equipamentos.
Diretriz	Projeto de pequeno/médio porte, com foco em backend Java, regras de negócio, persistência e fluxo de trabalho.
Stack obrigatória	Java 25 + Spring Boot 4.1.1 + Spring Web + Spring Data JPA + JPA (Jakarta Persistence) + Hibernate ORM + PostgreSQL + Angular + TypeScript + Angular Material; configuração inicial gerada pelo Spring Initializr com Maven, JAR e application.properties.

1. Visão geral

O ServiceFlow é um sistema web para pequenas assistências técnicas registrarem clientes, equipamentos, ordens de serviço, diagnósticos, orçamento e andamento do reparo. A ideia é representar um problema comercial real sem exigir estoque, emissão fiscal, pagamentos ou integrações externas.

Problema

Muitas pequenas assistências controlam entrada de equipamentos, orçamento, aprovação do cliente e andamento do serviço por papel, planilhas ou mensagens, dificultando rastreabilidade e comunicação interna.

Objetivo

Centralizar a ordem de serviço desde a entrada do equipamento até a entrega, mantendo histórico de status, diagnóstico, orçamento e informações do cliente.

Por que este projeto é adequado ao portfólio

O domínio permite demonstrar CRUDs necessários, mas o valor técnico está no fluxo de estados, regras de aprovação, modelagem relacional, transações, filtros, histórico e testes.

2. Escopo

2.1 Incluído

- Cadastro e autenticação de usuários internos.
- Papéis ADMIN e ATENDENTE.
- Cadastro de clientes.
- Cadastro de equipamentos vinculados a clientes.
- Abertura e acompanhamento de ordens de serviço.
- Registro de diagnóstico e observações técnicas.
- Criação de orçamento com valor estimado.
- Aprovação ou recusa do orçamento.
- Fluxo de status da ordem.
- Histórico de status e responsável pela alteração.
- Pesquisa por cliente, equipamento, código e status.
- Resumo operacional simples no dashboard.
- API REST com OpenAPI/Swagger.
- PostgreSQL com JPA (Jakarta Persistence), Hibernate ORM e Spring Data JPA.
- Testes automatizados e Docker Compose.

2.2 Fora do escopo da versão 1.0

- Controle de estoque e peças.
- Pagamento online.
- Emissão de nota fiscal.
- Integração com WhatsApp/e-mail.
- Assinatura digital.
- Aplicativo mobile nativo.
- Multiempresa com isolamento de tenants.
- Microserviços.

3. Atores e permissões

Ator	Responsabilidades
ADMIN	Gerenciar usuários e possuir acesso total às ordens, clientes, equipamentos e técnicos.
ATENDENTE	Cadastrar clientes/equipamentos, abrir ordens e atualizar informações operacionais autorizadas.
TÉCNICO (registro, não login na V1)	Responsável técnico identificado dentro da ordem; não precisa de conta própria na primeira versão.
CLIENTE (fora do login na V1)	Pessoa ou empresa proprietária do equipamento; seus dados são registrados para atendimento.

4. Conceitos do domínio

Conceito	Definição
Cliente	Pessoa ou empresa que solicita o serviço.
Equipamento	Bem do cliente que será analisado/reparado.
Ordem de serviço	Registro principal do atendimento, com equipamento, problema relatado, diagnóstico e andamento.
Orçamento	Valor estimado do serviço que precisa ser aprovado antes do início do reparo.
Histórico de status	Registro imutável das mudanças de situação da ordem.
Código da OS	Identificador legível, único e pesquisável da ordem de serviço.
Técnico	Profissional que executa o serviço e pode ser atribuído a uma ordem de serviço.

5. Requisitos funcionais

ID	Requisito	Descrição
RF-01	Cadastro de usuário	ADMIN cadastra usuário interno com nome, e-mail, senha e papel.
RF-02	Autenticação	Usuário interno realiza login e recebe JWT.
RF-03	Clientes	Criar, consultar, editar e listar clientes.
RF-04	Equipamentos	Criar, consultar, editar e listar equipamentos vinculados a um cliente.
RF-05	Abrir OS	Criar ordem com código, cliente, equipamento, data de entrada, problema relatado e prioridade.
RF-06	Diagnóstico	Registrar diagnóstico, solução proposta e observações técnicas.
RF-07	Orçamento	Criar/alterar orçamento associado à OS, com valor e observação.
RF-08	Aprovação	Registrar aprovação, recusa ou cancelamento do orçamento.
RF-09	Andamento	Alterar status da OS segundo as transições permitidas.
RF-10	Histórico	Registrar cada mudança de status com usuário, data e observação.
RF-11	Pesquisa	Pesquisar OS por código, cliente, equipamento e status, com paginação.
RF-12	Detalhe	Consultar visão consolidada da OS com cliente, equipamento, diagnóstico, orçamento e histórico.

ID	Requisito	Descrição
RF-13	Dashboard	Exibir quantidade de OS por status, prioridades e ordens abertas/aguardando aprovação.
RF-14	Desativação	Permitir desativação lógica de clientes e equipamentos quando houver histórico.
RF-15	Documentação	Disponibilizar OpenAPI/Swagger para todos os endpoints relevantes.
RF-16	Técnicos	ADMIN pode cadastrar, consultar, editar e desativar técnicos; técnicos desativados não podem ser atribuídos a novas OS.

6. Regras de negócio

ID	Regra	Descrição
RN-01	Código único	Cada OS possui código único no formato definido pelo sistema, por exemplo OS-2026-000001.
RN-02	Cliente obrigatório	Uma OS deve estar vinculada a exatamente um cliente.
RN-03	Equipamento obrigatório	Uma OS deve estar vinculada a exatamente um equipamento.
RN-04	Coerência do equipamento	O equipamento informado na OS deve pertencer ao cliente selecionado.
RN-05	Orçamento prévio	O serviço não pode entrar em EM_EXECUCAO antes de existir orçamento aprovado, exceto se a OS for marcada como SERVICIO_SEM_ORCAMENTO por uma regra explicitamente implementada. Na V1, recomenda-se exigir aprovação.
RN-06	Aprovação	Somente OS com orçamento em PENDENTE podem ter orçamento aprovado ou recusado.
RN-07	Transições	Mudanças de status devem respeitar o fluxo definido e não podem ocorrer em OS encerrada.
RN-08	Valor	Valor de orçamento não pode ser negativo.
RN-09	Condição de entrega	Uma OS só pode ser marcada como PRONTA quando o serviço estiver concluído.
RN-10	Entrega	Uma OS só pode ser marcada como ENTREGUE quando estiver PRONTA.
RN-11	Histórico imutável	Registros de histórico não podem ser editados pela API comum.

ID	Regra	Descrição
RN-12	Autorização	ATENDENTE só pode operar recursos aos quais o seu papel dá acesso; ADMIN pode supervisionar todos.
RN-13	Consistência transacional	Alterações que mudem status, orçamento e histórico devem ocorrer em transação.
RN-14	Concorrência	Atualizações simultâneas da mesma OS devem ser tratadas com controle de versão otimista ou outra estratégia documentada.

7. Fluxo da ordem de serviço

RECEBIDA -> EM_DIAGNOSTICO -> AGUARDANDO_APROVACAO

↓
 APROVADA -> EM_EXECUCAO -> PRONTA -> ENTREGUE
 | -> RECUSADA

CANCELADA pode ocorrer antes de PRONTA conforme as regras de negócio.

O projeto deve implementar a máquina de estados no service/domain, e não em controllers. A API deve rejeitar estados impossíveis, como ENTREGUE -> EM_EXECUCAO.

8. Casos de uso principais

Caso	Pré-condição	Fluxo principal	Resultado
UC-01 Abrir OS	Cliente e equipamento cadastrados.	Selecionar cliente -> equipamento -> problema -> abrir.	OS RECEBIDA com código único.
UC-02 Diagnosticar	OS RECEBIDA.	Registrar diagnóstico -> alterar status.	OS EM_DIAGNOSTICO.
UC-03 Enviar orçamento	Diagnóstico registrado.	Informar valor -> salvar orçamento -> enviar para aprovação.	OS AGUARDANDO_APROVACAO.
UC-04 Aprovar orçamento	Orçamento PENDENTE.	Registrar aprovação.	OS APROVADA.
UC-05 Executar serviço	Orçamento aprovado.	Iniciar reparo -> concluir -> marcar pronta.	OS PRONTA.
UC-06 Entregar	OS PRONTA.	Registrar entrega.	OS ENTREGUE.

9. Modelo de dados lógico

Entidade	Principais atributos	Relacionamentos
Usuario	id, nome, email, senha_hash, role, ativo, created_at, updated_at	1:N com Orcamento; 1:N com HistoricoStatusOS
Cliente	id, tipo, nome_razao, documento, email, telefone, ativo, created_at, updated_at	1:N com Equipamento; 1:N com OrdemServico
Tecnico	id, nome, especialidade, telefone, ativo, created_at	1:N com OrdemServico
Equipamento	id, cliente_id, tipo, marca, modelo, numero_serie, descricao, ativo, created_at, updated_at	N:1 com Cliente; 1:N com OrdemServico

Entidade	Principais atributos	Relacionamentos
OrdemServico	id, codigo, cliente_id, equipamento_id, tecnico_id, prioridade, status, problema_relatoado, diagnostico, solucao, data_entrada, data_inicio, data_pronta, data_entrega, version, created_at, updated_at	N:1 Cliente; N:1 Equipamento; N:1 Tecnico opcional; 1:N Orcamento; 1:N HistoricoStatusOS
Orcamento	id, ordem_id, numero_versao, valor, observacao, status, criado_por, decidido_at, created_at, updated_at	N:1 OrdemServico; N:1 Usuario
HistoricoStatusOS	id, ordem_id, usuario_id, status_anterior, status_novo, observacao, created_at	N:1 OrdemServico; N:1 Usuario

Persistência com JPA/Hibernate

As entidades devem ser mapeadas com Jakarta Persistence e persistidas pelo Hibernate ORM. Spring Data JPA cuidará dos repositórios. O atributo version da OrdemServico pode ser mapeado com @Version para implementar bloqueio otimista e demonstrar cuidado com concorrência sem transformar o projeto em um sistema distribuído.

10. API REST proposta

Método	Endpoint	Função	Acesso
POST	/api/auth/login	Login/JWT	Público
GET	/api/customers	Listar clientes	Autenticado
POST	/api/customers	Criar cliente	ATENDENTE/ADMIN
GET	/api/customers/{id}	Detalhe	Autenticado
PUT	/api/customers/{id}	Editar	ATENDENTE/ADMIN
DELETE	/api/customers/{id}	Desativar	ADMIN
GET	/api/customers/{id}/devices	Equipamentos do cliente	Autenticado
POST	/api/devices	Cadastrar equipamento	ATENDENTE/ADMIN
PUT	/api/devices/{id}	Editar equipamento	ATENDENTE/ADMIN
POST	/api/service-orders	Abrir OS	ATENDENTE/ADMIN
GET	/api/service-orders	Listar/pesquisar OS	Autenticado
GET	/api/service-orders/{id}	Detalhe consolidado	Autenticado
PATCH	/api/service-orders/{id}/status	Alterar status	ATENDENTE/ADMIN
PUT	/api/service-orders/{id}/diagnosis	Registrar diagnóstico	ATENDENTE/ADMIN
POST	/api/service-orders/{id}/quotes	Criar orçamento	ATENDENTE/ADMIN
PATCH	/api/quotes/{id}/approve	Aprovar	ATENDENTE/ADMIN
PATCH	/api/quotes/{id}/reject	Recusar	ATENDENTE/ADMIN
GET	/api/service-orders/{id}/history	Histórico	Autenticado
GET	/api/dashboard	Resumo operacional	ADMIN/ATENDENTE
GET	/api/technicians	Listar técnicos	ATENDENTE/ADMIN
POST	/api/technicians	Cadastrar técnico	ADMIN
PUT	/api/technicians/{id}	Editar técnico	ADMIN

Método	Endpoint	Função	Acesso
DELETE	/api/technicians/{id}	Desativar técnico	ADMIN

11. Stack, tecnologias e configuração inicial

Camada	Tecnologia	Uso no projeto
Linguagem	Java 25	Linguagem principal do backend.
Framework	Spring Boot 4.1.1	API REST, injeção de dependências, configuração e infraestrutura da aplicação.
Persistência	JPA (Jakarta Persistence) + Hibernate ORM	Mapeamento objeto-relacional das entidades e operações de persistência.
Acesso a dados	Spring Data JPA	Repositórios, paginação e abstração das operações de consulta.
Banco	PostgreSQL	Persistência relacional e integridade dos dados.
Segurança	Spring Security + JWT	Autenticação e autorização.
Validação	Bean Validation / Jakarta Validation	Validação dos dados de entrada e regras simples de consistência.
Testes	JUnit 5 + Mockito + Spring Boot Test	Testes unitários, integração e regras de negócio.
Documentação	OpenAPI / Swagger	Documentação e teste dos endpoints.
Build	Maven	Dependências, testes e empacotamento.
Infraestrutura	Docker Compose	Execução reproduzível da aplicação e do PostgreSQL.
Versionamento	Git + GitHub	Histórico, organização e portfólio.
Frontend	Angular + TypeScript + Angular Material	Aplicação web administrativa, navegação, Reactive Forms e componentes de interface.
HTTP no frontend	Angular HttpClient	Comunicação com a API REST e interceptors para autenticação/erros.

11.1 Configuração gerada pelo Spring Initializr

A configuração inicial do backend do ServiceFlow deve ser reproduzida a partir dos dados informados no Spring Initializr em 21/09/2026. *Esses valores representam o bootstrap do projeto e devem permanecer registrados para manter rastreabilidade da criação do projeto.*

Campo	Configuração
Project	Maven
Linguagem	Java
Spring Boot	4.1.1

Group	com.thiagoferreira
Artifact	ServiceFlow
Package name	com.thiagoferreira.ServiceFlow
Packaging	JAR
Configuration	Properties (application.properties)
Java	25

11.2 Dependências selecionadas no Spring Initializr

Spring Web — construção da API web REST com Spring MVC e servidor embarcado.

Spring Data JPA — persistência com Jakarta Persistence e integração com Hibernate.

PostgreSQL Driver — driver JDBC para conexão da aplicação com PostgreSQL.

H2 Database — banco em memória para testes e cenários de desenvolvimento controlados; não substitui o PostgreSQL como banco principal do projeto.

11.3 Dependências adicionais planejadas

Spring Validation — validação dos DTOs e dados recebidos pela API.

Spring Security — autenticação e autorização por papel.

JWT — mecanismo de autenticação stateless previsto para a API.

Flyway — versionamento e execução controlada das migrations do PostgreSQL.

OpenAPI/Swagger — documentação interativa dos endpoints da API.

Dependências de testes do ecossistema Spring/JUnit — testes unitários e de integração.

12. Requisitos não funcionais

ID	Categoria	Requisito
RNF-01	Java LTS	O backend deve ser desenvolvido em Java 25, conforme a configuração registrada no Spring Initializr e no README.
RNF-02	Spring Boot	A API deve usar Spring Boot com organização por camadas e injeção de dependências.
RNF-03	JPA + Hibernate	As entidades devem ser mapeadas com Jakarta Persistence (JPA) e persistidas com Hibernate ORM como provedor JPA.

ID	Categoria	Requisito
RNF-04	Spring Data JPA	Os repositórios devem utilizar Spring Data JPA; consultas customizadas só devem ser criadas quando necessárias.
RNF-05	PostgreSQL	A persistência deve usar PostgreSQL com chaves estrangeiras, restrições e índices necessários.
RNF-06	Segurança	Senhas com hash seguro, autenticação JWT e autorização server-side.
RNF-07	Validação	Entradas inválidas devem ser rejeitadas com mensagens claras e códigos HTTP adequados.
RNF-08	Testes	Regras críticas devem possuir testes unitários; endpoints e persistência críticos devem possuir testes de integração.
RNF-09	API	A API deve seguir convenções REST razoáveis e possuir documentação OpenAPI/Swagger.
RNF-10	Reprodutibilidade	Docker Compose deve permitir subir banco e aplicação com configuração documentada.
RNF-11	Qualidade	README, nomes, commits e estrutura de pacotes devem permitir que outra pessoa entenda o projeto sem explicação oral.
RNF-12	Privacidade	O repositório não deve conter senhas, tokens, chaves ou dados pessoais reais.
RNF-13	Consulta	Listagens de OS devem usar paginação e filtros por índice/consulta adequada.
RNF-14	Concorrência	A entidade OrdemServico deve permitir uma estratégia explícita de controle de concorrência, preferencialmente @Version na V1.
RNF-15	Dinheiro	Valores monetários devem usar BigDecimal; nunca double/float para persistência ou cálculo de preço.
RNF-16	Datas	Datas e horários devem usar java.time e política de timezone registrada no README.

13. Arquitetura sugerida

A primeira versão deve utilizar um monólito modular em camadas, criado com Maven pelo Spring Initializr e empacotado como JAR. O backend será desenvolvido em Java 25 com Spring Boot 4.1.1, Spring Web, Spring Data JPA, JPA (Jakarta Persistence) e Hibernate ORM. O PostgreSQL será o banco principal, com H2 reservado para testes e cenários controlados. O objetivo é demonstrar organização

de código e separação de responsabilidades sem introduzir complexidade desnecessária de microsserviços. O frontend é uma aplicação Angular separada, consumindo exclusivamente a API REST.

```
ServiceFlow

controller
  ↓
service / domain
  ↓
repository (Spring Data JPA)
  ↓
PostgreSQL

security / validation / exception handler

JPA (especificação) -> Hibernate ORM (implementação)
```

- Controller: recebe HTTP, valida DTOs básicos, chama serviços e monta respostas.
- Service: concentra casos de uso, regras de negócio e fronteiras transacionais.
- Domain/Entity: classes persistentes mapeadas com Jakarta Persistence.
- Repository: acesso ao banco por Spring Data JPA; consultas específicas somente quando necessário.
- DTO: separar contratos da API das entidades persistentes quando isso melhorar segurança e manutenção.
- Security: autenticação, geração/validação de JWT e autorização por usuário/papel.
- Exception handler: respostas de erro padronizadas e sem stack trace para o cliente.
- Hibernate: utilizado como implementação/provedor ORM de JPA; o código de domínio persistente deve ser modelado com Jakarta Persistence, enquanto o acesso aos repositórios deve usar Spring Data JPA. Evitar depender de funcionalidades proprietárias do Hibernate sem justificativa.
- O uso de @Version na entidade OrdemServico é recomendado para demonstrar controle otimista de concorrência de forma simples e justificável.

14. Tratamento de erros

```
{
  "timestamp": "2026-09-21T12:00:00Z",
  "status": 409,
  "code": "INVALID_STATUS_TRANSITION",
  "message": "A ordem não pode ser alterada de ENTREGUE para EM_EXECUCAO.",
  "path": "/api/service-orders/15/status"
}
```

Utilizar @RestControllerAdvice ou mecanismo equivalente para padronizar erros de validação, recurso inexistente, autorização, conflito de versão e violação de regra de negócio.

15. Testes e critérios de aceite

ID	Caso	Resultado esperado
CT-01	Equipamento pertence a outro cliente	Criação da OS rejeitada.
CT-02	Valor negativo no orçamento	Requisição rejeitada.

ID	Caso	Resultado esperado
CT-03	Aprovar orçamento inexistente/PENDENTE não	Operação rejeitada.
CT-04	Iniciar reparo sem aprovação	Operação rejeitada.
CT-05	OS ENTREGUE recebe novo status	Operação rejeitada.
CT-06	Código de OS duplicado	Inserção rejeitada.
CT-07	Histórico de status	Cada mudança gera registro imutável.
CT-08	Cliente/equipamento desativado	Não pode ser usado para nova OS.
CT-09	Concorrência com @Version	Atualização conflitante é detectada.
CT-10	Consulta paginada	Resposta retorna conteúdo e metadados de paginação.
CT-11	Autorização	Usuário sem papel adequado recebe erro de acesso.
CT-12	API documentada	Endpoints principais aparecem no Swagger.
Critério de pronto A versão 1.0 está pronta quando uma assistência fictícia consegue cadastrar cliente/equipamento, abrir uma OS, registrar diagnóstico, gerar orçamento, aprovar, executar e entregar o serviço, com histórico, testes e documentação funcionando.		

16. Plano de implementação enxuto

1. Bootstrap do Spring Boot 4.1.1 pelo Spring Initializr, Maven, Java 25, JAR, application.properties, PostgreSQL, H2 para testes, Docker e configuração por ambiente.
2. Usuários, autenticação JWT e autorização.
3. Entidades Cliente, Tecnico e Equipamento com JPA/Hibernate.
4. Entidade OrdemServico, código único e fluxo básico de status.
5. Diagnóstico, orçamento, aprovação/recusa e regras de transição.
6. Histórico de status e controle de concorrência com @Version.
7. Filtros/paginação, dashboard simples e tratamento de erros.
8. Testes unitários/integrados, OpenAPI, README, DER e publicação no GitHub.

17. Checklist de portfólio

- README orientado a problema de negócio, não apenas a tecnologia.
- Diagrama ER e descrição das entidades.
- Diagrama da máquina de estados da OS.
- Exemplos de regras de negócio e respostas de erro.
- Swagger/OpenAPI funcional.
- Testes de fluxo completo e de regras críticas.
- Docker Compose funcional.
- Maven, .env.example e instruções de execução.
- Commits pequenos e descritivos.
- Sem dados reais de clientes.

18. Modelo de dados completo

Esta seção detalha o modelo relacional de referência do ServiceFlow. A V1 continua pequena, mas todas as entidades persistentes, tabelas, chaves e relacionamentos necessários para o fluxo de ordem de serviço estão definidos.

18.1 Visão geral das tabelas

#	Tabela	Entidade JPA	Finalidade
1	usuarios	Usuario	Contas internas e permissões.
2	clientes	Cliente	Pessoa ou empresa atendida pela assistência.
3	tecnicos	Tecnico	Profissionais responsáveis pela execução técnica; não precisam autenticar na V1.
4	equipamentos	Equipamento	Equipamentos pertencentes aos clientes.
5	ordens_servico	OrdemServico	Registro central do atendimento e ciclo do reparo.
6	orcamentos	Orcamento	Valores e decisões de orçamento, permitindo revisões.
7	historicos_status_os	HistoricoStatusOS	Auditoria de cada transição da ordem.

18.2 Tabela usuarios

Coluna	Tipo PostgreSQL	Nulo?	Chave/Regra	Descrição
id	BIGINT	Não	PK	Identificador.
nome	VARCHAR(120)	Não	-	Nome do usuário interno.
email	VARCHAR(160)	Não	UNIQUE	E-mail de login.
senha_hash	VARCHAR(255)	Não	-	Senha armazenada como hash.
role	VARCHAR(20)	Não	ENUM lógico	ADMIN ou ATENDENTE.
ativo	BOOLEAN	Não	DEFAULT TRUE	Conta habilitada.
created_at	TIMESTAMP WITH TIME ZONE	Não	-	Criação.
updated_at	TIMESTAMP WITH TIME ZONE	Não	-	Última alteração.

Constraint principal: UNIQUE(email). Índice opcional em (role, ativo).

18.3 Tabela clientes

Coluna	Tipo PostgreSQL	Nulo?	Chave/Regra	Descrição
id	BIGINT	Não	PK	Identificador do cliente.

Coluna	Tipo PostgreSQL	Nulo?	Chave/Regra	Descrição
tipo	VARCHAR(10)	Não	ENUM lógico	PF ou PJ.
nome_razao	VARCHAR(160)	Não	-	Nome completo ou razão social.
documento	VARCHAR(20)	Sim	UNIQUE quando informado	CPF/CNPJ formatado somente na apresentação, normalizado na persistência.
email	VARCHAR(160)	Sim	-	E-mail de contato.
telefone	VARCHAR(30)	Sim	-	Telefone de contato.
ativo	BOOLEAN	Não	DEFAULT TRUE	Permite desativação lógica.
created_at	TIMESTAMP WITH TIME ZONE	Não	-	Criação.
updated_at	TIMESTAMP WITH TIME ZONE	Não	-	Última alteração.

18.4 Tabela tecnicos

Coluna	Tipo PostgreSQL	Nulo?	Chave/Regra	Descrição
id	BIGINT	Não	PK	Identificador do técnico.
nome	VARCHAR(120)	Não	-	Nome do técnico.
especialidade	VARCHAR(120)	Sim	-	Área principal de atuação.
telefone	VARCHAR(30)	Sim	-	Contato interno.
ativo	BOOLEAN	Não	DEFAULT TRUE	Permite retirar técnico da operação sem apagar histórico.
created_at	TIMESTAMP WITH TIME ZONE	Não	-	Criação.

A entidade Tecnico existe para evitar armazenar o nome do profissional diretamente na ordem de serviço e permitir consultas por técnico.

18.5 Tabela equipamentos

Coluna	Tipo PostgreSQL	Nulo?	Chave/Regra	Descrição
id	BIGINT	Não	PK	Identificador.
cliente_id	BIGINT	Não	FK -> clientes.id	Proprietário do equipamento.
tipo	VARCHAR(60)	Não	-	Notebook, celular, impressora etc.
marca	VARCHAR(80)	Não	-	Marca.
modelo	VARCHAR(100)	Sim	-	Modelo.
numero_serie	VARCHAR(120)	Sim	-	Número de série, quando existir.
descricao	TEXT	Sim	-	Características relevantes.

Coluna	Tipo PostgreSQL	Nulo?	Chave/Regra	Descrição
ativo	BOOLEAN	Não	DEFAULT TRUE	Indica se pode receber novas ordens.
created_at	TIMESTAMP WITH TIME ZONE	Não	-	Criação.
updated_at	TIMESTAMP WITH TIME ZONE	Não	-	Última alteração.

Índices: cliente_id; opcionalmente numero_serie. Equipamento desativado não pode ser usado para nova OS.

18.6 Tabela ordens_servico

Coluna	Tipo PostgreSQL	Nulo?	Chave/Regra	Descrição
id	BIGINT	Não	PK	Identificador interno.
codigo	VARCHAR(30)	Não	UNIQUE	Código público da OS, ex.: OS-2026-000001.
cliente_id	BIGINT	Não	FK -> clientes.id	Cliente da ordem.
equipamento_id	BIGINT	Não	FK -> equipamentos.id	Equipamento atendido.
tecnico_id	BIGINT	Sim	FK -> tecnicos.id	Técnico responsável na execução.
prioridade	VARCHAR(20)	Não	ENUM lógico	BAIXA, MEDIA, ALTA ou URGENTE.
status	VARCHAR(30)	Não	ENUM lógico	Estado atual da OS.
problema_relatoado	TEXT	Não	-	Descrição do defeito informado pelo cliente.
diagnostico	TEXT	Sim	-	Diagnóstico técnico.
solucao	TEXT	Sim	-	Descrição do serviço executado.
data_entrada	TIMESTAMP WITH TIME ZONE	Não	-	Entrada do equipamento.
data_inicio	TIMESTAMP WITH TIME ZONE	Sim	-	Início do reparo.
data_pronta	TIMESTAMP WITH TIME ZONE	Sim	-	Momento em que ficou pronta.
data_entrega	TIMESTAMP WITH TIME ZONE	Sim	-	Momento da entrega.
version	BIGINT	Não	@Version	Controle otimista de concorrência.
created_at	TIMESTAMP WITH TIME ZONE	Não	-	Criação da OS.
updated_at	TIMESTAMP WITH TIME ZONE	Não	-	Última alteração.

A FK equipamento_id deve permanecer coerente com cliente_id. Essa regra é validada no service antes da criação/alteração da OS.

18.7 Tabela orcamentos

Coluna	Tipo PostgreSQL	Nulo?	Chave/Regra	Descrição
id	BIGINT	Não	PK	Identificador do orçamento.
ordem_id	BIGINT	Não	FK -> ordens_servico.id	Ordem relacionada.

Coluna	Tipo PostgreSQL	Nulo?	Chave/Regra	Descrição
numero-versao	INTEGER	Não	>= 1	Versão/revisão do orçamento.
valor	NUMERIC(12,2)	Não	>= 0	Valor total estimado.
observacao	TEXT	Sim	-	Detalhes do orçamento.
status	VARCHAR(20)	Não	ENUM lógico	PENDENTE, APROVADO ou RECUSADO.
criado_por	BIGINT	Não	FK -> usuarios.id	Usuário que registrou o orçamento.
decidido_at	TIMESTAMP WITH TIME ZONE	Sim	-	Momento da aprovação/recusa.
created_at	TIMESTAMP WITH TIME ZONE	Não	-	Criação.
updated_at	TIMESTAMP WITH TIME ZONE	Não	-	Atualização.

Constraints: UNIQUE(ordem_id, numero-versao); CHECK(valor >= 0); no máximo um orçamento PENDENTE por ordem.

18.8 Tabela historicos_status_os

Coluna	Tipo PostgreSQL	Nulo?	Chave/Regra	Descrição
id	BIGINT	Não	PK	Identificador do evento.
ordem_id	BIGINT	Não	FK -> ordens_servico.id	OS alterada.
usuario_id	BIGINT	Não	FK -> usuarios.id	Usuário que realizou a transição.
status_anterior	VARCHAR(30)	Sim	ENUM lógico	Status anterior; nulo somente na criação.
status_novo	VARCHAR(30)	Não	ENUM lógico	Novo status.
observacao	TEXT	Sim	-	Motivo ou observação da mudança.
created_at	TIMESTAMP WITH TIME ZONE	Não	-	Momento do evento.

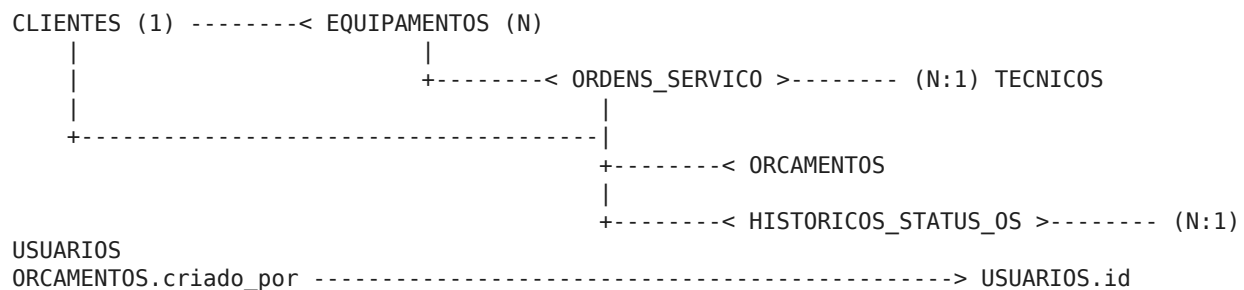
Histórico append-only; não permitir edição pela API comum.

18.9 Entidades JPA e relacionamentos

Entidade	Relacionamento	Cardinalidade	Mapeamento JPA de referência	Descrição
Cliente	Cliente -> Equipamento	1:N	@OneToMany(mappedBy = "cliente")	Cliente pode possuir vários equipamentos.
Equipamento	Equipamento -> Cliente	N:1	@ManyToOne(fetch = LAZY)	Cada equipamento possui um cliente proprietário.
Cliente	Cliente -> OrdemServico	1:N	@OneToMany(mappedBy = "cliente")	Cliente pode possuir várias ordens ao longo do tempo.

Entidade	Relacionamento	Cardinalidade	Mapeamento JPA de referência	Descrição
Equipamento	Equipamento -> OrdemServico	1:N	@OneToMany(mappedBy = "equipamento")	Equipamento pode aparecer em várias ordens ao longo do tempo.
Tecnico	Tecnico -> OrdemServico	1:N	@OneToMany(mappedBy = "tecnico")	Um técnico pode ser atribuído a várias OS.
OrdemServico	OrdemServico -> Cliente	N:1	@ManyToOne(fetch = LAZY)	Cada OS possui exatamente um cliente.
OrdemServico	OrdemServico -> Equipamento	N:1	@ManyToOne(fetch = LAZY)	Cada OS atende exatamente um equipamento.
OrdemServico	OrdemServico -> Tecnico	N:1 opcional	@ManyToOne(fetch = LAZY)	Pode não haver técnico atribuído no momento da abertura.
OrdemServico	OrdemServico -> Orcamento	1:N	@OneToMany(mappedBy = "ordem")	Uma OS pode ter revisões de orçamento.
Orcamento	Orcamento -> Usuario	N:1	@ManyToOne(fetch = LAZY)	Cada orçamento registra quem o criou.
OrdemServico	OrdemServico -> HistoricoStatusOS	1:N	@OneToMany(mappedBy = "ordem")	Uma OS possui histórico de status.
HistoricoStatusOS	HistoricoStatusOS -> Usuario	N:1	@ManyToOne(fetch = LAZY)	Cada evento registra o autor.
Usuario	Usuario -> OrdemServico	1:N	via criado/atualizado conforme auditoria	Usuário opera ordens, sem FK de autor obrigatório na OS V1.

18.10 Diagrama relacional textual



FKs: equipamentos.cliente_id -> clientes.id; ordens_servico.cliente_id -> clientes.id;
ordens_servico.equipamento_id -> equipamentos.id; ordens_servico.tecnico_id -> tecnicos.id;
orcamentos.ordem_id -> ordens_servico.id; orcamentos.criado_por -> usuarios.id;
historicos_status_os.ordem_id -> ordens_servico.id; historicos_status_os.usuario_id -> usuarios.id.

18.11 Estratégia JPA/Hibernate

- Todas as entidades devem utilizar Jakarta Persistence (@Entity, @Id, @ManyToOne, @OneToMany e constraints apropriadas) e Hibernate ORM como provedor JPA.
- Spring Data JPA deve ser utilizado para repositories, paginação e consultas de persistência usuais.
- Enums de status, prioridade e papel devem ser persistidos com @Enumerated(EnumType.STRING).
- Relacionamentos N:1 devem utilizar fetch LAZY e coleções também devem ser mantidas LAZY; evitar EAGER por padrão.

- A entidade OrdemServico deve possuir @Version no atributo version para detectar atualização concorrente.
- A criação de orçamento, mudança de status e inclusão de histórico devem ocorrer dentro de boundaries transacionais no service.
- Use BigDecimal para valores monetários e mapeie o valor como NUMERIC(12,2) no PostgreSQL.
- Entidades não devem ser usadas como DTOs de resposta em endpoints complexos; utilizar DTOs para controlar campos expostos e evitar problemas de serialização de relacionamentos.
- Evitar CascadeType.ALL em relacionamentos que envolvam histórico; o histórico deve sobreviver às alterações da ordem.

18.12 Integridade e exclusão

Objeto	Política
Usuario	Desativação lógica com ativo=false; preservar ordens e históricos existentes.
Cliente	Desativação lógica quando houver histórico; não apagar registros que possuam equipamentos/OS.
Tecnico	Desativação lógica; ordens antigas preservam o vínculo.
Equipamento	Desativação lógica; impedir uso em novas OS.
OrdemServico	Não excluir fisicamente na V1; preservar histórico operacional.
Orcamento	Preservar versões e decisões; não permitir DELETE pela API comum.
HistoricoStatusOS	Append-only; somente INSERT.

19. Contrato de implementação do banco

- As migrations do PostgreSQL devem ser versionadas no Git e executadas automaticamente na inicialização controlada do ambiente de desenvolvimento.
- A aplicação deve funcionar com banco vazio a partir das migrations, sem depender de alterações manuais do schema.
- FKs, UNIQUEs, CHECKs e índices simples devem ser definidos no banco para reforçar as invariantes que não dependem de contexto de negócio.
- A sequência de código público da OS deve ser gerada de forma segura e não pode depender de um contador mantido apenas em memória.
- Dados fictícios de demonstração devem ser separados da estrutura das migrations e nunca conter informações reais de clientes.

20. Critérios de aceite do modelo de dados

- As sete tabelas existem no PostgreSQL com PKs, FKs e constraints documentadas.
- As entidades Java correspondem às tabelas por JPA/Hibernate e os relacionamentos estão mapeados explicitamente.
- As regras de status, orçamento, unicidade, autorização e concorrência possuem testes de integração.
- Listagens são paginadas e o histórico permanece imutável.

21. Frontend - Angular + TypeScript

O frontend do ServiceFlow deve ser uma aplicação web responsiva desenvolvida em Angular com TypeScript. O objetivo é oferecer uma interface administrativa de uso diário para uma assistência técnica fictícia, com foco em ordens de serviço, clientes, equipamentos, técnicos, orçamentos e histórico.

Tecnologia definida: Angular + TypeScript. A navegação deve utilizar Angular Router. A comunicação com a API deve utilizar HttpClient. Formulários devem utilizar Reactive Forms. Angular Material pode ser adotado para acelerar a construção de uma interface administrativa consistente sem criar componentes visuais complexos do zero.

21.1 Objetivos do frontend

- Permitir autenticação de usuários internos e restringir recursos por papel.
- Disponibilizar uma visão operacional das ordens de serviço.
- Reduzir a quantidade de telas necessárias para executar o atendimento de ponta a ponta.
- Permitir pesquisa e filtragem de ordens, clientes e equipamentos.
- Apresentar claramente o status atual e o histórico da ordem.
- Permitir registrar diagnóstico, orçamento, atribuição de técnico e evolução do serviço.
- Manter o frontend desacoplado dos detalhes de persistência do backend.

21.2 Navegação e rotas

Rota	Tela	Acesso
/login	Login	Público
/dashboard	Dashboard	ADMIN/ATENDENTE
/clientes	Lista de clientes	ADMIN/ATENDENTE
/clientes/novo	Novo cliente	ADMIN/ATENDENTE
/clientes/:id	Detalhes do cliente	ADMIN/ATENDENTE
/equipamentos/novo	Novo equipamento	ADMIN/ATENDENTE
/ordens	Lista de ordens de serviço	ADMIN/ATENDENTE
/ordens/nova	Abertura de OS	ADMIN/ATENDENTE
/ordens/:id	Detalhe da OS	ADMIN/ATENDENTE
/ordens/:id/orcamento	Orçamento	ADMIN/ATENDENTE
/tecnicos	Gestão de técnicos	ADMIN
/usuarios	Gestão de usuários	ADMIN

21.3 Telas detalhadas

ID	Tela	Comportamento	Elementos principais
FE-01	Login	Autenticar usuário interno via POST /api/auth/login e bloquear recursos sem sessão.	E-mail, senha, botão Entrar, mensagens de erro, loading.
FE-02	Dashboard	Exibir resumo operacional consumindo os dados agregados pela API.	Cards de OS por status, prioridades, aguardando aprovação e abertas; atalhos para nova OS e listagem.
FE-03	Clientes	Listar clientes com busca, paginação e acesso ao detalhe.	Tabela; busca; status ativo/inativo; paginação; ações.
FE-04	Cadastro/edição de cliente	Criar ou editar cliente com validações de documento, e-mail e telefone quando informados.	Tipo PF/PJ; nome/razão social; documento; e-mail; telefone; ativo.
FE-05	Detalhe do cliente	Exibir dados do cliente e seus equipamentos e ordens relacionadas.	Dados cadastrais; equipamentos; últimas OS; ações para novo equipamento/nova OS.
FE-06	Cadastro/edição de equipamento	Vincular equipamento a cliente e permitir alterações autorizadas.	Tipo; marca; modelo; número de série; descrição; status ativo.
FE-07	Ordens de serviço	Pesquisar e filtrar OS por código, cliente, equipamento, status, prioridade e técnico.	Tabela; filtros; paginação; badges de status/prioridade; acesso ao detalhe.
FE-08	Abertura de OS	Criar ordem com cliente, equipamento, problema relatado, prioridade e data de entrada.	Seleção encadeada de cliente/equipamento; validação de coerência; confirmação de abertura.

ID	Tela	Comportamento	Elementos principais
FE-09	Detalhe da OS	Centralizar todos os dados operacionais da ordem.	Dados do cliente/equipamento; problema; diagnóstico; solução; técnico; status; orçamento; histórico.
FE-10	Diagnóstico e execução	Permitir registrar diagnóstico, solução proposta e evolução conforme permissões.	Campos técnicos; ações de mudança de status; confirmação antes de transições críticas.
FE-11	Orçamento	Criar nova versão, consultar versões e registrar decisão conforme a API.	Número da versão; valor; observação; status; decisão; histórico de versões.
FE-12	Histórico da OS	Exibir linha do tempo imutável dos estados da ordem.	Data/hora; usuário; status anterior; novo status; observação.
FE-13	Técnicos	ADMIN cadastra, edita, consulta e desativa técnicos.	Nome; especialidade; telefone; ativo; confirmação de desativação.
FE-14	Usuários	ADMIN gerencia contas internas e papéis.	Nome; e-mail; papel; ativo; confirmação de desativação.

21.4 Requisitos funcionais do frontend

ID	Requisito	Descrição
FE-RF-01	Autenticação	O frontend deve autenticar e manter o estado do usuário conforme a estratégia definida para o token.
FE-RF-02	Guards	Rotas devem utilizar guards para impedir acesso não autenticado e acesso incompatível com o papel.

ID	Requisito	Descrição
FE-RF-03	Filtros e paginação	Listagens de OS e clientes devem suportar filtros e paginação sem carregar todos os registros.
FE-RF-04	Seleção relacionada	Na abertura da OS, o equipamento deve ser carregado conforme o cliente selecionado e a coerência final deve ser validada pelo backend.
FE-RF-05	Status	A interface deve exibir somente ações compatíveis com o status atual, sem substituir a validação do backend.
FE-RF-06	Orçamento	A tela de orçamento deve suportar versões e apresentar claramente a decisão atual.
FE-RF-07	Concorrência	Se a API retornar conflito de versão/atualização, a interface deve informar que a OS foi alterada por outro usuário e orientar nova consulta.
FE-RF-08	Erros 401/403/409	Respostas de autenticação, autorização e conflito devem possuir tratamento visual específico.
FE-RF-09	Feedback	Toda operação de criação, alteração, status e desativação deve apresentar confirmação visual.
FE-RF-10	Responsividade	As telas administrativas devem continuar utilizáveis em notebooks e telas menores, priorizando reorganização de tabelas e formulários.

21.5 Componentes e serviços reutilizáveis

Item	Responsabilidade
AuthGuard	Impedir acesso a rotas privadas.

Item	Responsabilidade
RoleGuard	Restringir rotas por ADMIN/ATENDENTE.
AuthInterceptor	Adicionar token às requisições e encaminhar respostas 401 conforme estratégia de sessão.
StatusBadge	Exibir status da OS e do orçamento de forma padronizada.
PriorityBadge	Exibir prioridade da OS.
PaginationControls	Paginação de listas.
ConfirmDialog	Confirmação de operações críticas.
ServiceOrderTimeline	Histórico de status.
CustomerForm / EquipmentForm / ServiceOrderForm	Formulários reutilizáveis com Reactive Forms.
ApiErrorInterceptor/handler	Tratamento visual padronizado de erros HTTP.

21.6 Integração frontend-backend

Service Angular	Endpoint(s)	Uso
AuthService	POST /api/auth/login	Autenticação.
CustomerService	GET/POST/PUT/DELETE /api/customers	Clientes.
EquipmentService	GET/POST/PUT/DELETE /api/equipment	Equipamentos.
ServiceOrderService	GET/POST/PUT/PATCH /api/service-orders	OS, filtros e transições.
QuoteService	Endpoints de orçamento definidos na API	Versões e decisões de orçamento.
TechnicianService	GET/POST/PUT/DELETE /api/technicians	Técnicos.
DashboardService	GET /api/dashboard ou endpoints agregados definidos na API	Indicadores.

Os serviços Angular devem encapsular HttpClient e converter as respostas da API para tipos TypeScript. As páginas devem coordenar componentes e serviços, mas não concentrar chamadas HTTP diretamente em múltiplos pontos.

21.7 Arquitetura do frontend

Estrutura sugerida: core -> shared -> features -> pages/components. Core deve conter autenticação, interceptors, guards e configuração. Shared deve conter componentes reutilizáveis. Cada feature (clientes, equipamentos, ordens, técnicos, usuários) deve agrupar suas telas, modelos e serviços relacionados.

Não é necessário introduzir NgRx na V1. O estado de telas administrativas pode ser controlado por componentes e serviços, utilizando observables/Signals quando trouxerem benefício claro.

21.8 Critérios de aceite do frontend

- Usuário interno consegue autenticar e acessar somente as telas permitidas pelo papel.
- Atendente consegue executar o fluxo de cadastro do cliente até a abertura da OS.
- Usuário consegue consultar, filtrar e paginar ordens de serviço.
- A tela de detalhe permite acompanhar diagnóstico, orçamento, status e histórico.
- As transições inválidas são bloqueadas pelo backend e tratadas adequadamente pelo frontend.
- Conflitos de concorrência retornam feedback compreensível ao usuário.
- O dashboard apresenta informações coerentes com os dados da API.
- As principais telas são responsivas e utilizáveis em resolução de notebook.

22. Fechamento e escopo de portfólio

O frontend faz parte do escopo da V1 e deve ser entregue funcionando junto ao backend Java. A implementação deve priorizar a jornada principal do usuário e a qualidade da integração com a API. Recursos cosméticos ou funcionalidades não essenciais podem ficar fora da V1, desde que o fluxo principal esteja completo, testado e documentado.

A divisão tecnológica definida para o portfólio é: Pet Connect com React + TypeScript no frontend e ServiceFlow com Angular + TypeScript no frontend. Ambos utilizam Java + Spring Boot no backend, PostgreSQL na persistência e Jakarta Persistence (JPA) com Hibernate ORM para mapeamento objeto-relacional.